COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	October 16, 2026	**DATE SUBMITTED:**	October 21, 2026
NAME:	Uchi, Mao Sophie S. Verano, James Daniel M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 12.1 Algorithmic Strategies

1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

Table 12-1. Algorithmic Strategies and Examples

Strategy	Algorithm	Analysis
Recursion	Tree Traversal Strategies (In-Order, Pre-Order, Post-Order) or Depth-First Search	This traversal/searching algorithm proceeds to visit each node in a particular order; traversal techniques are usually known for its linear structure that has a singular path of locating an element, but with these algorithms, more than one path can be visited. **A recursion can be seen in this algorithm by the strategies and search function itself, until the base case is achieved**.
Brute Force	Sequential / Linear Search	This algorithm is straightforward and blatant on what this search could offer in order to match the strategy; a Linear Search works on O(n) time complexity, comparing each element chronologically until the designated element is found. So what makes this search Brute Force? **This search has absolutely tackled every solution that results in finding the target element, one-by-one comparisons, until each element in a list is compared from beginning to end**.
Backtracking	Depth-First Search	This search algorithm continues to approach branches until all unvisited nodes are called, but the additional function that makes Depth-First Search and Breadth-First Search algorithm different is that **Depth-First Search has the ability to turn back**

		searching from its current path to a different path, seeking the unexplored nodes left.
Greedy	Selection Sort	This sorting algorithm begins searching for the smallest element in the array to be compared with the first element, making the first index sorted. Then seek for the second smallest element to be compared with the second index; repeat these steps until the array is sorted. **What this sort achieves as a Greedy Strategy, the decision to swap the minimum elements, forcing to lock in the smaller elements into its final position, but in risk of error that could fail to recognize if the element is truly lesser**.
Divide-and-Conquer	Quick Sort	This sorting algorithm starts off by using the Divide-and-Conquer approach, and picking a pivot element from an array to find the middle, separating the array; left partition with the first subarray and right partition with second subarray. Making use of the pivot, the left partition will be compared until it is sorted; smaller elements from the left partition and greater elements from the right partition. Pick another pivot and repeat the process until the left partition is sorted, do it again with the right partition; merge both subarrays to complete the array. So what makes this algorithm Divide-and-Conquer? **This sort is self-explanatory, operating with the strategy itself by subdividing the problem into plenty of smaller parts, solving it easier with this method**.

Table 12-2. Memoization Implementation

Screenshot	int memo[n+1]; // we will initialize the elements to -1 (-1 means, not solved it yet)
	int getMinSteps (int n)
	{
	if (n == 1) return 0; // base case
	if(memo[n] != -1) return memo[n]; // we have solved it already :)
	int r = 1 + getMinSteps(n - 1); // '-1' step . 'r' will contain the optimal answer finally
	if(n%2 == 0) r = min(r , 1 + getMinSteps(n / 2)) ; // '/2' step
	if(n%3 == 0) r = min(r , 1 + getMinSteps(n / 3)) ; // '/3' step
	memo[n] = r ; // save the result. If you forget this step, then its same as plain recursion.

	return r; }
Analysis	In this implementation, its **Top-Down Dynamic Programming (Memoization)** strategy is presented to solve the minimum steps problem efficiently. In the first initialization, which is the **memo array**, it's initialized to -1 to keep track of unsolved state as well mentioned in the comment clarifying its purpose. Before the recursive calls begin, it has specific function conditions that checks first the subproblem. In the first if condition, if n is equivalent to 1, the program will end its execution since it met the condition. In the next if condition **memo[n] != 1**, this **validates that if a solution already exists**, it returns it instantly, therefore avoiding a lot of recursion calls taking up space complexity. Otherwise, it will proceed to compute the value recursively, find the most minimal steps among the valid operations, and store the result in the array before returning. With this algorithm technique, this will help programmers optimize the time complexity from exponential to linear case scenario by eliminating redundant overlapping subproblems.

Table 12-3. Bottom-Up Dynamic Programming Implementation

Screenshot	int getMinSteps (int n)
	{
	int dp[n+1], i;
	dp[1] = 0; // trivial case
	for(i = 2 ; i < = n ; i ++)
	{
	dp[i] = 1 + dp[i-1];
	if(i%2==0) dp[i] = min(dp[i] , 1+ dp[i/2]);
	if(i%3==0) dp[i] = min(dp[i] , 1+ dp[i/3]);
	}
	return dp[n];
	}

Analysis	**Bottom-Up Dynamic Programming** also known as **(Tabulation)** strategy. This implementation determines the minimum steps to reach 1. Comparing the memoization, this approach fully eliminates recursion and the risk of call stack overflow. It works **iteratively** from the base case, which is the difference compared to memoization. It iterates from the base case up to the target value which is **n**. It uses a **for loop** to build solutions for each subsequent index i, where in, as it iterates for every n, it calculates the cost of subtracting 1 and compares it against the options of dividing 2 or 3 (whether its divisible) to select the absolute minimum step count. This implementation has a high optimal execution time of **O(n)** and utilizes memory space with its iterative approach to find the most efficient step count.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies &
ILO C: Create C++ code to solve problems using learned algorithmic strategies

Problem Title: Count the number of paths in a matrix with a given cost to reach the destination cell

Description:
Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).

For example, consider the following 4 × 4 matrix where cost is 25. Two paths are having a cost of 25. A sample path has been given when the cost is 25.

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

The idea is to use recursion. The problem has optimal substructure. That means the problem can be broken down into smaller, simple “subproblems”, which can further be divided into yet simpler, smaller subproblems until the solution becomes trivial.

Deliverables:

Pseudocode

```
FUNCTION Paths(matrix, m, n, cost):
    IF cost <, 0 THEN
        RETURN 0
    ENDIF

    IF m == 0 AND n == 0 THEN
        IF matrix[0][0] == cost THEN
            RETURN 1
        ELSE
            RETURN 0
        ENDIF
    ENDIF

    IF m == 0 THEN
```

```
        RETURN Paths(matrix, 0, n - 1, cost - matrix[m][n])
    ENDIF

    IF n == 0 THEN
        RETURN Paths(matrix, m - 1, 0, cost - matrix[m][n])
    ENDIF

    RETURN Paths(matrix, m - 1, n, cost - matrix[m][n]) +
                Paths(matrix, m, n - 1, cost - matrix[m][n])
ENDFUNCTION

INT main() {
    INITIALIZE int M = matrix.size();
    INITIALIZE int N = matrix[0].size();

    FOR int target = 20; target <= 30; target++:
        INITIALIZE pathFound = Paths(matrix, M - 1, N - 1, target
        IF pathFound > 0 THEN
            DISPLAY “Paths Found.”
        ELSE
            DISPLAY “0 Paths Found.”
    RETURN 0
}
```

(+ Solving Problem by Hand)

	(0, 0)	(0, 1)	(0, 2)	(0, 3)
(0 ,0)	4	7	1	6
(1, 0)	6	7	3	9
(2, 0)	3	8	1	2
(3, 0)	7	1	7	3

Following the created layout above:

Since in the pseudocode, we decided to use a range of target cost to check what

	are the cost values that actually has a valid path or existing path throughout the summation of its direction to (3, 3). By that, we only limited the array range cost value between 20 to 30. And here are the only outcomes we have solved by hand through coordinates. **Target cost = 21 | Total Path(s): 1** **Coordinates:** (0, 0) -> (0, 1) -> (0, 2) -> (1, 2) -> (2, 2) -> (2, 3) -> (3, 3) **Calculation:** 4 + 7 + 1 + 3 + 1 + 2 + 3 = **21** **Target cost = 26 | Total Path(s): 2** **Coordinates [1]:** (0, 0) -> (1, 0) -> (1, 1) -> (1, 2) -> (2, 2) -> (2, 3) -> (3, 3) **Calculation:** 4 + 6 + 7 + 3 + 1 + 2 + 3 = **26** **Coordinates [2]:** (0, 0) -> (0, 1) -> (0, 2) -> (1, 2) -> (2, 2) -> (3, 2) -> (3, 3) **Calculation:** 4 + 7 + 1 + 3 + 1 + 7 + 3 = **26** **Target cost = 27 | Total Path(s): 2** **Coordinates [1]:** (0, 0) -> (1, 0) -> (2, 0) -> (2, 1) -> (2, 2) -> (2, 3) -> (3, 3) **Calculation:** 4 + 6 + 3 + 8 + 1 + 2 + 3 = **27** **Coordinates [2]:** (0, 0) -> (0, 1) -> (1, 1) -> (1, 2) -> (2, 2) -> (2, 3) -> (3, 3) **Calculation:** 4 + 7 + 7 + 3 + 1 + 2 + 3= **27** **Target cost = 29 | Total Path(s): 1** **Coordinates:** (0, 0) -> (0, 1) -> (0, 2) -> (1, 2) -> (1, 3) -> (2, 3) -> (3, 3) **Calculation:** 4 + 7 + 1 + 3 + 9 + 2 + 3 = **29** The rest of the target cost, which are the 20, 22, 23, 24, 25, 28, and 30, are identified as an impossible cost to calculate as the algorithm couldn't find a valid pathway or sum combination that will equate to the given costs. Since it has a strict condition, it is bound to be limited. As it recursives, it tries every possible combination combined with the dynamic programming, yet with the condition, it's impossible.
Working C++ code	

Code:

```cpp
#include <iostream>
#include <vector>

int Paths(const std::vector<std::vector<int>>& matrix, int m, int n, int cost) {
    // first condition, no negative values.
    if (m < 0 || n < 0 || cost < 0) {
        return 0;
    }

    // second condition, Once in the starting cell, calculations
    // and checks the sum of the path if it is equivalent to
    // the cost (target value)
    if (m == 0 && n == 0) {
        return (matrix[m][n] == cost) ? 1 : 0;
    }

    // third condition, If m has reached 0, It means it's in the
    // first row / top row. Meaning it can only move left as a
    // valid path to reach (0, 0)
    if (m == 0) {
        return Paths(matrix, 0, n - 1, cost - matrix[m][n]);
    }

    // last condition, The opposite of the m, if n has reached 0, it
    // memans it's in the first column or left most of the matrix, therefore,
    // it can only move up as a valid path to reach (0, 0)
    if (n == 0) {
        return Paths(matrix, m - 1, 0, cost - matrix[m][n]);
    }

    // if no condition has been met yet, it will keep recursing until
    // it reaches the end of its condition.
    int fromTop = Paths(matrix, m - 1, n, cost - matrix[m][n]);
    int fromLeft = Paths(matrix, m, n - 1, cost - matrix[m][n]);

    return fromTop + fromLeft;
}
```

```cpp
int main() {
    std::vector<std::vector<int>> matrix = {
        {4, 7, 1, 6},
        {6, 7, 3, 9},
        {3, 8, 1, 2},
        {7, 1, 7, 3}
    };

    // represents the number of columns
    int M = matrix.size();

    // represents the number of rows
    int N = matrix[0].size();
```

```cpp
    // lookups a target cost from 20 - 30. verifying all possible paths.
    for (int target = 20; target <= 30; target++) {
        int pathFound = Paths(matrix, M - 1, N - 1, target);

        std::cout << " Target Cost [" << target << "] : ";
        if (pathFound > 0) {
            std::cout << pathFound << " Path(s) Found in the matrix." << std::endl;
        }
        else {
            std:: cout << "0 paths found in the matrix." << std::endl;
        }

    }
    return 0;
}
```

Output:

```
Target Cost [20] : 0 paths found in the matrix.
Target Cost [21] : 1 Path(s) Found in the matrix.
Target Cost [22] : 0 paths found in the matrix.
Target Cost [23] : 0 paths found in the matrix.
Target Cost [24] : 0 paths found in the matrix.
Target Cost [25] : 0 paths found in the matrix.
Target Cost [26] : 2 Path(s) Found in the matrix.
Target Cost [27] : 2 Path(s) Found in the matrix.
Target Cost [28] : 0 paths found in the matrix.
Target Cost [29] : 1 Path(s) Found in the matrix.
Target Cost [30] : 0 paths found in the matrix.
```

Analysis of working code

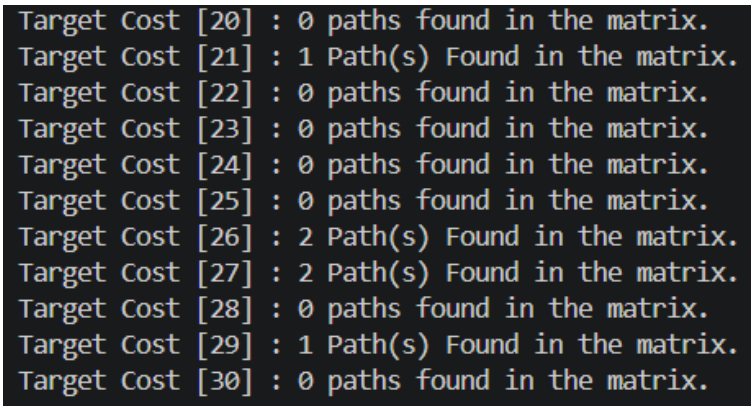
Uchi's Analysis:

From the code above, this algorithm is a top-down recursive strategy with optimal substructure. Instead of looking for paths at the start, we will begin in reverse, from the destination cell to the source cell which is the vertex(0,0) inside the 4x4 matrix; looking for a combination that has the exact target cost ranging from 20 to 30.

Located at the top of the program, we see the function 'Path'; there are four implemented IF conditions to manage and reduce the error on finding a valid path. First IF condition checks the variable whether it is less than or equal to zero, avoiding any non-positive integers. Second IF condition checks the matrix if it stops ahead at the starting cell, the matrix's path is already equal to the cost and should return true. Third and fourth IF conditions commands the code to limit the matrix as it is prohibited to enter a nonexistent vertex, exceeding out of the maximum row and columns of the matrix result in error as the path is not valid. Therefore, if any of the conditions are not used, the program may proceed, computing for every available combination until one of the conditions is met.

Verano's Analysis:

In the code implementation of the supplementary, I used a top-down recursive strategy that uses and **represents optimal substructure** by **breaking the matrix path problem into more manageable subproblems**. As the code represents, My way of implementation is instead of tracking paths from the beginning, **the algorithm works backward from the destination cell towards the source cell (0, 0)**. The backtracking works via a recursion, to inspect every viable / valid path combination. The main function executes a loop containing different conditions validating the **target cost ranging from 20 to 30**. I improvised the scenario since aiming for 25 only, with the experimental approach, it seems that there's no valid path for a target cost of 25, therefore outputting a 0, which doesn't actually help if the algorithm actually works. Therefore, I created the inspection of possible path combinations ranging from 20 to 30 against a 4 x 4 cost matrix.

	First let's expound the conditions inside the function so that we will be able to break it down one by one clearly. In the first if condition **if (m < 0 || n < 0 || cost < 0)**, is a condition where their program will end if any of the variables for inspection and validation is less than or equal to zero. This is a condition to prevent the program from actually functioning a non positive integer that may crash the program. In the next if condition, **if(m == 0 && n == 0)**, this is the condition wherein the matrix has already reached the starting cell, and the matrix's calculated path is equivalent to the cost (target value), it shall **return true**. Otherwise, **false**. In the last 2 if conditions, which is the **if (m == 0)** and **if (n == 0)**, these are the conditions that control the limit of the matrix so that it won't go out of bound. If m has already reached the index value of 0, therefore, you cannot go up anymore since it's already the max top row of the matrix. Otherwise, if n is equivalent to 0, you cannot go to the left anymore since you've reached the left most of the matrix. As no condition can be met yet, the recursion will keep going until one condition has been met and the program has finished all iterations. Otherwise, it will keep calculating possible path combination up until the program has reached its final condition to end the program.
Screenshots of demonstration	

Uchi's Conclusion

My conclusion in this activity, this module focuses on our ability to recognize and have an understanding towards different algorithmic strategies which are recursion, brute force, backtracking, greedy, divide-and-conquer and dynamic programming. It is plausible that each of these algorithmic strategies had been encountered from the previous activities, so it is upon us if we don't comprehend this before, but alas, I had remembered what and which algorithm fits the description of each strategy. But there are some strategies I'm unfamiliar with such as

memoization, which is an optimization strategy that speeds up the programs by thinking of ways on how to improve and increase its time efficiency. From my partner's program, I was stunned to know this is how you should apply memoization, since I don't have better fundamentals when thinking of different approaches on improving a program as I am stuck with the usual basic computation at hand.

Verano's Conclusion

Overall, in doing this hands-on activity, It was a challenge since matrices are one confusing structure to try and implement especially in data structure and algorithm. I gained a more comprehensive understanding of each algorithmic strategy such as **recursion, brute force, backtracking, greedy, divide-and-conquer** and **dynamic programming**. With some of them familiar, I was able to easily adapt in implementing it towards the code. In the **memoization** and **bottom-up tabulation**, with various ways of algorithmic strategies to solve complex problems, integrating recursion with dynamic programming totally helps resolve the redundant calculation and high time complexities of the programs. With dynamic programming, it helped me to be more creative to build solutions iteratively and improve program performance and resource management (space complexity). With expounding more of the algorithmic strategy, I have seen myself to still be at the far level of being a future **computer engineer** given how I am still struggling in data handling.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Google AI Overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concept, grammar correction.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The matrix scenario is interpreted for implementations.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.